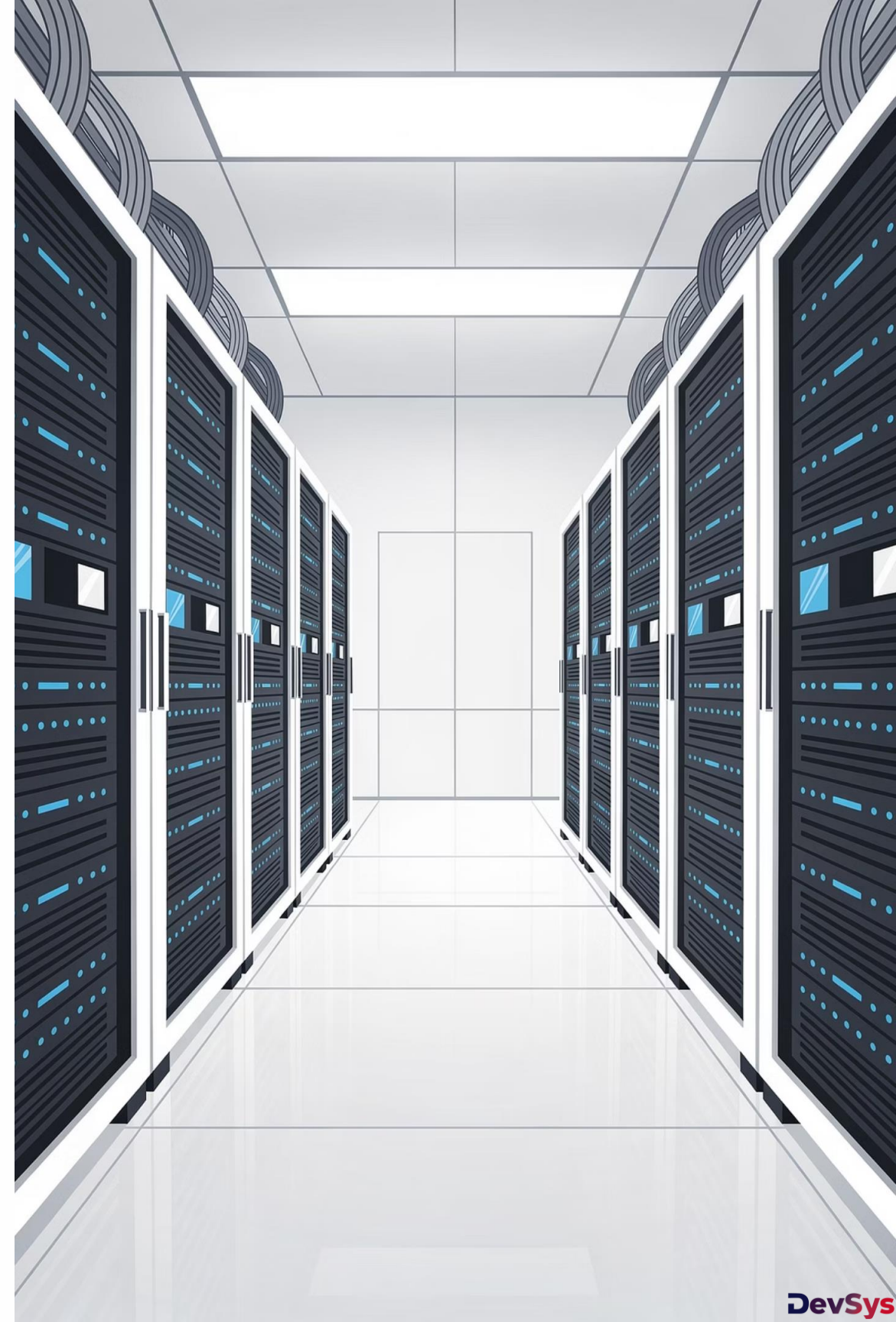


REST Fayl Əməliyyatları və Content Negotiation

Spring Web çərçivəsində fayl yükləmə, endirmə və Content Negotiation mexanizmlərini öyrənəcəyik. Bu dərs həm yeni başlayanlar, həm də irəli səviyyə tələbələr üçün nəzərdə tutulmuşdur.

SPRING WEB

DƏRSLİK 10



Bu Dərstdə Nə Öyrənəcəyik?

01

Fayl Upload

Client-dən server-ə fayl göndərmə əməliyyatı —
MultipartFile istifadəsi.

02

Fayl Download

Server-dən client-ə fayl göndərmə — Resource
və ResponseEntity istifadəsi.

03

Content Negotiation

Client-in istədiyi formata uyğun cavab vermə
mexanizmi — Accept header, produces,
consumes.

Real Dünya Ssenariləri

Bu mövzu gündəlik backend işlərinin əsasını təşkil edir. Aşağıdakı hallar hamısı bu dərsin mövzusuna aiddir:

📄 Profil Şəkli

İstifadəçi öz şəklini sistemə yükləyir — **upload** əməliyyatı.

📄 PDF Endirmə

Sistem sənədi istifadəçiyə göndərir — **download** əməliyyatı.

📊 Excel İxrac

Hesabat faylı olaraq ixrac edilir — **export** əməliyyatı.

File Handling Nədir?

Tərif

Backend vasitəsilə fayllarla işləmək deməkdir. İki əsas istiqamət var: faylları **qəbul etmək** (upload) və faylları **göndərmək** (download).

Upload vs Download

Upload: Client → Server istiqaməti. İstifadəçi faylı serverə göndərir.

Download: Server → Client istiqaməti. Server faylı istifadəçiyə göndərir.

Multipart Request Nədir?

Fayl göndərmək üçün xüsusi HTTP sorğu növü istifadə olunur. Bu sorğunun **Content-Type** dəyəri `multipart/form-data` olur.

Niyə Multipart?

Adi JSON sorğusu ikili (binary) fayl məlumatını daşıya bilmir. Multipart format həm mətn, həm də fayl məlumatını eyni sorğuda göndərməyə imkan verir.

Content-Type Başlığı

Sorğu göndərilərkən başlıqda `Content-Type: multipart/form-data` göstərilməlidir. Postman bunu avtomatik əlavə edir.

Spring-də Fayl Qəbul Etmək

Spring-də fayl qəbul etmək üçün `MultipartFile` interfeysi istifadə olunur. Endpoint `@PostMapping` ilə işarələnir, parametr isə `@RequestParam` ilə alınır.

Annotasiya

`@PostMapping("/upload")` – POST sorğusunu qəbul edir.

Parametr

`@RequestParam MultipartFile file` – faylı parametr kimi alır.

Nəticə

`file.getOriginalFilename()` – faylın orijinal adını qaytarır.

```
@PostMapping("/upload")
public String upload(@RequestParam MultipartFile file) {
    return file.getOriginalFilename();
}
```

MultipartFile Nədir?

Spring-in `MultipartFile` interfeysi yüklənən faylı təmsil edən obyektidir. Bu obyekt vasitəsilə faylın bütün məlumatlarına çatmaq mümkündür.



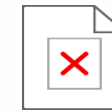
Fayl Adı

`getOriginalFilename()` — faylın orijinal adını qaytarır.



Fayl Ölçüsü

`getSize()` — faylın bayt ölçüsünü qaytarır.



Fayl Məzmunu

`getBytes()` — faylın ikili məzmununu qaytarır.

Faylı Diskə Yazmaq

Qəbul edilən faylı serverə saxlamaq üçün `transferTo()` metodu istifadə olunur. Fayl göstərilən yola yazılır.

```
file.transferTo(new File("uploads/" + file.getOriginalFilename()));
```



`new File("uploads/...")` – hədəf yolu göstərilir.



`transferTo()` – faylı həmin yola yazır.



Fayl server üzərindəki **uploads/** qovluğunda saxlanır.



Qovluğun mövcud olduğundan əmin olun, əks halda `IOException` xətası baş verər.

Upload Praktiki Axını



Client göndərir

Backend qəbul edir

Diskə yazır

Response qaytarır

Bu ardıcılıq hər fayl yükləmə əməliyyatının əsasını təşkil edir. Hər addım düzgün işləməzsə, fayl saxlanmaz.

Fayl Download Endpoint

Fayl endirmə üçün `ResponseEntity<Resource>` qaytaran GET endpoint yazılır. `Resource` interfeysi faylı təmsil edir.

```
@GetMapping("/download")
public ResponseEntity<Resource> download() {
    File file = new File("uploads/test.txt");
    Resource resource = new FileSystemResource(file);
    return ResponseEntity.ok()
        .header("Content-Disposition",
            "attachment; filename=test.txt")
        .body(resource);
}
```

FileSystemResource

Diskdəki faylı `Resource` obyektinə çevirir. Spring bu obyekti birbaşa cavab gövdəsi kimi göndərə bilir.

Content-Disposition

`attachment; filename=...` başlığı brauzerin faylı endirməsini təmin edir, ekranda göstərmir.

Content-Type Nədir?

HTTP cavabında `Content-Type` başlığı göndərilən məlumatın növünü bildirir. Server bu başlıq vasitəsilə client-ə "bu məlumat hansı formatdadır" deyir.

`application/json`

JSON formatında məlumat. REST API-lərdə ən çox istifadə olunan növdür.

`text/plain`

Sadə mətn formatı. Düz yazı faylları üçün istifadə olunur.

`application/pdf`

PDF sənəd formatı. Hesabat və sənəd endirmələrində istifadə olunur.

`multipart/form-data`

Fayl yükləmə sorğuları üçün xüsusi format. Binar məlumat daşıyır.

Content Negotiation Nədir?

Tərif

Client hansı formatda cavab istədiyini bildirir, server isə uyğun formatda cavab verir. Bu mexanizm **Accept** başlığı vasitəsilə işləyir.

Necə İşləyir?

1. Client sorğuda **Accept: application/json** göndərir.
2. Server bu başlığı oxuyur.
3. Server JSON formatında cavab qaytarır.

Əgər server istənilən formatı dəstəkləmərsə, **406 Not Acceptable** xətası qaytarır.

JSON vs XML Müqayisəsi

**JSON — müasir
sistemlər
oxunaqlı
yüngül
REST API-lərdə
standart**



**XML — köhnə
sistemlər
verbose
ağır
enterprise
sistemlərdə
istifadə olunur**



Müasir REST API-lərdə JSON standart formatdır. XML isə köhnə enterprise sistemlərlə integrasiyada hələ də istifadə olunur.

produces və consumes istifadəsi

Spring-də endpoint-in hansı formatı qəbul edib qaytaracağını `produces` və `consumes` atributları ilə müəyyən edirik.

produces — Qaytarılan Format

Endpoint-in hansı formatda cavab verəcəyini göstərir.

```
@GetMapping(value = "/students",  
            produces = "application/json")
```

consumes — Qəbul Edilən Format

Endpoint-in hansı formatda sorğu qəbul edəcəyini göstərir.

```
@PostMapping(value = "/upload",  
            consumes = "multipart/form-data")
```

Postman ilə Test Etmək

⬆️ Upload Testi

1. Postman-da POST sorğusu seç.
2. Body → **form-data** seç.
3. Key sahəsinə `file` yaz, növünü **File** et.
4. Value sahəsindən fayl seç.
5. Sorğunu göndər, cavabı yoxla.

⬇️ Download Testi

1. Postman-da GET sorğusu seç.
2. URL-i daxil et: `/download`
3. Sorğunu göndər.
4. Cavabda fayl avtomatik endirilir.

Brauzer ilə də test etmək mümkündür — URL-i birbaşa daxil edin.

Təhlükəsizlik Məsələləri

Fayl yükləmə funksiyası düzgün qorunmasa, ciddi təhlükəsizlik boşluqlarına yol açə bilər. Aşağıdakı tədbirlər mütləq tətbiq edilməlidir:

-  **Fayl Ölçüsünü Limitlə**
`spring.servlet.multipart.max-file-size=5MB` — böyük faylların serverə yüklənməsinin qarşısını alır.
-  **Fayl Tipini Yoxla**
Yalnız icazə verilən uzantıları (`.jpg`, `.pdf` və s.) qəbul et. `.exe` kimi zərərli faylları rədd et.
-  **Fayl Adını Sanitize Et**
Orijinal fayl adını birbaşa istifadə etmə. UUID ilə unikal ad ver, upload qovluğunu ayrı saxla.

Best Practice – Ən Yaxşı Təcrübələr

1

Unikal Ad Ver

`UUID.randomUUID()` istifadə edərək hər fayla unikal ad ver. Eyni adlı faylların üzərinə yazılmasının qarşısını alır.

2

Ayrı Qovluq

Upload edilən faylları layihə qovluğundan kənarda saxla. Məsələn:
`/var/uploads/`

3

Xəta İdarəetməsi

Fayl əməliyyatlarını `try-catch` bloku ilə əhat et. İstifadəçiyə aydın xəta mesajı qaytar.

4

Konfiqurasiya

`application.properties`-də multipart limitlərini mütləq təyin et. Default dəyərlər çox kiçik ola bilər.

Ən Çox Edilən Səhvlər

⚠️ Multipart Konfigurasiya

`spring.servlet.multipart.enabled=true` unutmaq. Fayl qəbul edilmir, 400 xətası gəlir.

⚠️ Fayl Yolu Səhvləri

Nisbi yol istifadə etmək. Qovluğun mövcud olmadığı halda `IOException` baş verir.

⚠️ Content-Type Səhvi

`produces/consumes` dəyərini yanlış yazmaq. 415 Unsupported Media Type xətasına səbəb olur.

Real Layihələrdə İstifadə

Bu dərsdə öyrəndiklərimiz real backend layihələrinin əsas funksiyalarıdır:



Avatar Upload

İstifadəçi profil şəklini yükləyir. Şəkil ölçüsü yoxlanılır, unikal adla saxlanır.



Sənəd Saxlama

Müqavilə, sertifikat kimi sənədlər sistemə yüklənir və lazım olduqda endirilir.



Hesabat İxracı

Sistem məlumatları Excel və ya PDF formatında ixrac edilir. Content-Type düzgün təyin edilir.

Nəticə və Xülasə

File Handling

Real backend-in əsas funksiyasıdır. `MultipartFile` ilə upload, `Resource` ilə download həyata keçirilir.

Content Negotiation

API çevikliyini təmin edir. `Accept` başlığı, `produces` və `consumes` atributları ilə idarə olunur.

Təhlükəsizlik

Fayl ölçüsü, tipi və adı mütləq yoxlanılmalıdır. Best practice-lərə riayət etmək vacibdir.

- 📄 Bu dərsin materiallarına əsasən quiz sualları hazırlanacaq. Bütün anlayışları — `MultipartFile`, `ResponseEntity`, `Content-Type`, `produces`, `consumes` — yaxşı mənimsəyin.